

SEMESTER: 5

COURSE NO: PCCST502

COURSE NAME: DESIGN
AND ANALYSIS OF
ALGORITHMS

L-T-P-R: 3:1:0:0

HOURS: 4

CREDIT: 4

NAME/S OF STAFF:

VIJI C



St. Thomas Institute for Science & Technology
Trivandrum, (College Code: STI)

COURSE MATERIAL

Department of Computer Science &
Engineering(DS)

Prepared by:

VIJI C

Verified by:

HOD

Approved by:

PRINCIPAL

COLLEGE VISION

To be a premier institution in the region providing quality technical education, to create competent professionals with good human and social values.

COLLEGE MISSION

1. To create a conducive environment for a quality learning, by providing a learner centric environment.
2. Offering higher level training in theory and practice, including human and social values.
3. Providing knowledge based service and skills to prepare professionals of tomorrow.

DEPARTMENT VISION

To be a leading department in Data Science Engineering, empowering students with cutting-edge technological skills and a commitment to ethical and social responsibilities, thereby fostering a vibrant learning environment.

DEPARTMENT MISSION

M1: To cultivate a deep understanding of both theoretical and practical aspects of Data Science Engineering.

M2: To create a learner-centric environment that promotes career-oriented education and innovation.

M3: To deliver high-quality education and training, preparing students to excel as data science professionals and responsible citizens

PROGRAMME EDUCATIONAL OBJECTIVES (PEO)

PEO1: Graduates shall evolve as globally competent Data Science professionals, capable of developing intelligent, data-driven solutions across multidisciplinary domains.

PEO2: Graduates shall acquire up-to-date knowledge in Data Science, Computer Science, and allied disciplines such as statistics, mathematics, and domain-specific sciences to build successful careers or pursue higher education.

PEO3: Graduates shall cultivate professional ethics, teamwork, leadership, and effective communication skills, enabling them to make socially responsible and impactful contributions to society.

PROGRAMME OUTCOME

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

PROGRAMME SPECIFIC OUTCOME (PSO)

PSO1: The students will be able to analyze, model, and interpret complex data to derive meaningful insights and make informed decisions.

PSO2: Enable the students to apply core concepts of Data Science, including data preprocessing, machine learning, and visualization, to build innovative and real-world solutions.

SYLLABUS

MODULE 1

Algorithms – Characteristics, Criteria for Analysing Algorithms; Time and Space Complexity - Best, Worst, and Average Case Complexities; Asymptotic Notations and their properties; Time and Space Complexity Calculation of simple algorithms; Analysis of Recursive Algorithms - Recurrence Equations, Solution of Recurrence Equations : Iteration Method, Recursion Tree Method, Substitution method and Master's Theorem (proof not expected); Balanced Search Trees - AVL Trees (Insertion and deletion operations with all rotations in detail, algorithms not expected)

MODULE 2

Disjoint Sets - Disjoint set operations, Union and find algorithms, Analysis of union by rank with path compression, Connected components of a Graph; Graphs – Representations, Traversals : BFS, DFS and their analysis, Strongly Connected Components; Topological Sorting. Divide and Conquer Strategy – Control Abstraction, Merge Sort, Strassen's Matrix Multiplication, Analysis.

MODULE 3

Greedy Strategy - Control Abstraction, Fractional Knapsack; Minimum Cost Spanning Tree – Kruskal's and Prim's, Analysis; Shortest Path Problem – Dijkstra's Algorithm, Analysis; Dynamic Programming - Control Abstraction, Optimality Principle, Matrix Chain Multiplication, Analysis; All Pairs Shortest Path Algorithm - Floyd-Warshall Algorithm, Analysis; Backtracking - Control Abstraction, N – Queens Problem, Algorithm.

MODULE 4

Branch and Bound - Control Abstraction, Travelling Salesman Problem, Algorithm; Complexity - Tractable and Intractable Problems; Complexity Classes : P, NP, NP- Hard and NP-Complete Classes; NP Completeness proof - Clique Problem and Vertex Cover Problem; Approximation algorithms - Bin Packing; Randomized Algorithms - Definitions of Monte Carlo and Las Vegas algorithms; Randomized version of Quick Sort algorithm with analysis.

TABLE OF CONTENTS

SL NO	TITLE	PAGE NO
Module 1		
1	Algorithms -Characteristics, Criteria for Analysing Algorithms	
2	Time and Space Complexity - Best, Worst, and Average Case Complexities	
3	Asymptotic Notations and their properties	
4	Time and Space Complexity Calculation of simple algorithms	
5	Analysis of Recursive Algorithms - Recurrence Equations, Solution of Recurrence Equations	
6	Iteration Method, Recursion Tree Method, Substitution method and Master's Theorem (proof not expected)	
7	Balanced Search Trees - AVL Trees (Insertion and deletion operations with all rotations in detail, algorithms not expected)	
Module II		
8	Disjoint Sets -Disjoint set operations, Union and find algorithms, Analysis of union by rank with path compression, Connected components of a Graph	
9	Graphs – Representations, Traversals	
10	BFS, DFS and their analysis, Strongly Connected Components	
11	Topological Sorting. Divide and Conquer Strategy – Control Abstraction, Merge Sort, Strassen's Matrix Multiplication, Analysis.	
Module-III		
12	Greedy Strategy -Control Abstraction, Fractional Knapsack	
13	Minimum Cost Spanning Tree – Kruskal's and Prim's, Analysis	
14	Shortest Path Problem – Dijkstra's Algorithm, Analysis	
15	Dynamic Programming - Control Abstraction, Optimality Principle, Matrix Chain Multiplication, Analysis	
16	All Pairs Shortest Path Algorithm - Floyd-Warshall Algorithm, Analysis	
17	Backtracking - Control Abstraction, N – Queens Problem, Algorithm.	
Module-IV		
18	Branch and Bound - Control Abstraction, Travelling Salesman Problem, Algorithm	
19	Complexity - Tractable and Intractable Problems	
20	Complexity Classes : P, NP, NP- Hard and NP-Complete Classes	
21	NP Completeness proof - Clique Problem and Vertex Cover Problem	
22	Approximation algorithms - Bin Packing;	
23	Randomized Algorithms - Definitions of Monte Carlo and Las Vegas algorithms	
24	Randomized version of Quick Sort algorithm with analysis.	

MODULE 1

Algorithms

An algorithm is a finite sequence of well-defined instructions used to solve a problem or perform a computation.

Characteristics of an Algorithm

Input – Accepts zero or more inputs.

Output – Produces at least one output.

Definiteness – Every step must be clearly defined.

Finiteness – Must terminate after a finite number of steps.

Effectiveness – Each step must be simple and executable.

Criteria for Analyzing Algorithms

Algorithm analysis helps determine efficiency.

Factors Considered

Time Complexity

- Measures execution time.
- Depends on input size.

Space Complexity

- Measures memory usage.
- Includes variables, data structures, and recursion stack.
-

Time Complexity

Best Case Complexity

Minimum time required by an algorithm.

Example:

Linear Search finds the element in the first position.

$$T(n) = O(1)$$

Worst Case Complexity

Maximum time required by an algorithm.

Example:

Linear Search finds the element at the last position.

$$T(n) = O(n)$$

Average Case Complexity

Expected time taken for all possible inputs.

Example:

Linear Search

$$T(n) = O(n)$$

Space Complexity

Space used by an algorithm during execution.

Formula

$\text{Space Complexity} = \text{Fixed Part} + \text{Variable Part}$
--

Fixed Part:

- Program instructions
- Constants
- Variables

Variable Part:

- Dynamic memory allocation
- Recursion stack

Example:

```
int sum(int n)
{
    int s=0;
    for(int i=1;i<=n;i++)
        s=s+i;
```

}

Space Complexity = $O(1)$

Asymptotic Notations

Used to express growth rate of algorithms.

Big O Notation $O(g(n))$

Represents upper bound.

Example:

$$T(n) = 3n^2 + 5n + 2$$

$$O(n^2)$$

Big Omega $\Omega(g(n))$

Represents lower bound.

Example:

$$T(n) = 3n^2 + 5n + 2$$

$$\Omega(n^2)$$

Big Theta $\Theta(g(n))$

Represents tight bound.

Example:

$$T(n) = 3n^2 + 5n + 2$$

$$\Theta(n^2)$$

Little o Notation

Represents strict upper bound.

Example:

$$n = o(n^2)$$

Little ω Notation

Represents strict lower bound.

Example:

$$n^2 = \omega(n)$$

Properties of Asymptotic Notations

Reflexive Property:- $f(n) = O(f(n))$

Transitive Property:- If $f(n) = O(g(n))$ and $g(n) = O(h(n))$ then $f(n) = O(h(n))$

Symmetry Property:- If $f(n) = \Theta(g(n))$ then $g(n) = \Theta(f(n))$

Time Complexity Calculation

Example 1

```
for(i=1;i<=n;i++)
{
    printf("Hello");
}
```

Number of executions = n

Time Complexity = $O(n)$

Example 2

```
for(i=1;i<=n;i++)
{
    for(j=1;j<=n;j++)
    {
        printf("Hello");
    }
}
```

Executions = $n \times n$

Time Complexity = $O(n^2)$

Example 3

```
for(i=1;i<=n;i*=2)
```

```
{
    printf("Hello");
}
```

Iterations = $\log_2 n$

Time Complexity = $O(\log n)$

Recursive Algorithms

A recursive algorithm calls itself until a base condition is reached.

Example:

```
factorial(n)
{
    if(n==0)
        return 1;
    else
        return n*factorial(n-1);
}
```

Recurrence Equations

Used to represent recursive algorithms.

General Form:

$$T(n) = aT(n/b) + f(n)$$

where

a = Number of recursive calls

b = Input reduction factor

f(n) = Extra work

Example:

$$T(n) = T(n-1) + 1$$

Solving Recurrence Relations

Iteration Method

Example:

$$T(n) = T(n-1) + 1$$

Expand:

$$T(n) = T(n-2) + 2$$

$$T(n) = T(n-3) + 3$$

...

$$T(n) = T(1) + (n-1)$$

Result:

$$T(n) = O(n)$$

Recursion Tree Method

Example:

$$T(n) = 2T(n/2) + n$$

Level 0 cost = n

Level 1 cost = n

Level 2 cost = n

...

Number of levels = $\log n$

Total Cost:

$n \log n$

Result:

$$T(n) = O(n \log n)$$

Substitution Method

Guess the solution and verify.

Example:

$$T(n)=T(n-1)+1$$

Assume:

$$T(n)=O(n)$$

Substitute and prove.

Result:

$$T(n)=O(n)$$

Master's Theorem

Used to solve recurrence relations of form:

$$T(n)=aT(n/b)+f(n)$$

Case 1

If

$$f(n)=O(n^{(\log_b a - \epsilon)})$$

Then

$$T(n)=\Theta(n^{(\log_b a)})$$

Case 2

If

$$f(n)=\Theta(n^{(\log_b a)})$$

Then

$$T(n)=\Theta(n^{(\log_b a)} \log n)$$

Case 3

If

$$f(n)=\Omega(n^{(\log_b a + \epsilon)})$$

Then

$$T(n)=\Theta(f(n))$$

Example 1

$$T(n)=2T(n/2)+1$$

$$a=2, b=2$$

$$n^{(\log_2 2)}=n$$

Result:

$$T(n)=\Theta(n)$$

Example 2

$$T(n)=2T(n/2)+n$$

Result:

$$T(n)=\Theta(n \log n)$$

Example 3

$$T(n)=2T(n/2)+n^2$$

Result:

$$T(n)=\Theta(n^2)$$

AVL Trees

AVL Tree is a self-balancing Binary Search Tree developed by:

- **Adelson-Velsky**
- **Landis**

For every node:

$$\text{Balance Factor} = \text{Height}(\text{Left Subtree}) - \text{Height}(\text{Right Subtree})$$

Balance Factor must be:

$$-1, 0, +1$$

Rotations in AVL Tree

LL Rotation (Single Right Rotation)

Occurs when insertion is made in left subtree of left child.

Before:

30

/

20

/

10

After:

20

/ \

10 30

RR Rotation (Single Left Rotation)

Occurs when insertion is made in right subtree of right child.

Before:

10

\

20

\

30

After:

20

/ \

10 30

LR Rotation (Double Rotation)

Occurs when insertion is made in right subtree of left child.

Before:

30

/

10

\

20

Step 1: Left Rotation at 10

Step 2: Right Rotation at 30

After:

20

/ \

10 30

RL Rotation (Double Rotation)

Occurs when insertion is made in left subtree of right child.

Before:

10

\

30

/

20

Step 1: Right Rotation at 30

Step 2: Left Rotation at 10

After:

20

/ \

10 30

AVL Tree Insertion

Steps

- 1. Insert node as BST.**
- 2. Calculate Balance Factor.**
- 3. If imbalance occurs:**
 - **LL → Right Rotation**
 - **RR → Left Rotation**
 - **LR → Left + Right Rotation**
 - **RL → Right + Left Rotation**

Time Complexity:

$O(\log n)$

AVL Tree Deletion

Steps

- 1. Delete node as BST.**
- 2. Update heights.**
- 3. Compute Balance Factor.**
- 4. Apply suitable rotations.**

Possible Rotations:

- **LL**
- **RR**
- **LR**
- **RL**

Time Complexity:

$O(\log n)$

Comparison: BST vs AVL Tree

Feature	BST	AVL Tree
Balance	Not Balanced	Self Balanced
Search	$O(n)$ Worst Case	$O(\log n)$
Insertion	$O(n)$ Worst Case	$O(\log n)$
Deletion	$O(n)$ Worst Case	$O(\log n)$
Rotations	Not Required	Required

MODULE 2

Disjoint Sets, Graph Algorithms & Divide and Conquer

Disjoint Sets (Union-Find Data Structure)

A Disjoint Set is a data structure used to maintain a collection of non-overlapping (disjoint) sets.

Applications:

- Kruskal's Minimum Spanning Tree
- Network connectivity
- Image processing
- Dynamic graph connectivity

Operations

1. MakeSet(x)

Creates a new set containing only element x.

```
MAKESET(x)
parent[x] = x
rank[x] = 0
```

Time Complexity:

O(1)

2. Find(x)

Finds the representative (root) of the set.

Without optimization

```
Find(x)
if parent[x] == x
    return x
return Find(parent[x])
```

Worst Case

1→2→3→4→5

Time Complexity

O(n)

3. Union(x,y)

Combines two sets.

```
Union(x, y)
```

```
root1 = Find(x)
```

```
root2 = Find(y)
```

```
parent[root2] = root1
```

Union by Rank

Instead of attaching arbitrarily, attach the smaller tree under the larger tree.

Rank = Approximate tree height.

Algorithm

Union(x,y)

```
rx = Find(x)
```

```
ry = Find(y)
```

```
if rank[rx] < rank[ry]
```

```
    parent[rx] = ry
```

```
else if rank[rx] > rank[ry]
```

```
    parent[ry] = rx
```

```
else
```

```
    parent[ry] = rx
```

```
    rank[rx]++
```

Advantages

- Tree height remains small
- Faster Find operation

Complexity

$O(\log n)$

Path Compression

Whenever Find() is called,

Make every visited node point directly to the root.

Before

```
1
|
2
|
3
|
4
|
5
```

After Find(5)

```
  1
 /  |  \
2  3  4  5
```

Algorithm

Find(x)

```
if parent[x] != x
    parent[x] = Find(parent[x])
```

```
return parent[x]
```

Union by Rank + Path Compression

Both optimizations together.

Complexity

$O(\alpha(n))$

where

$\alpha(n)$ = Inverse Ackermann Function

Practically constant time.

Connected Components using Disjoint Sets

Connected Component

A maximal set of connected vertices.

Algorithm

For every vertex

 MakeSet(v)

For every edge(u,v)

 Union(u,v)

For every vertex

 Find(v)

Time Complexity

$O(E \alpha(V))$

Graph Representation

Graph

A ----- B

| |
| |

C ----- D

1. Adjacency Matrix

A B C D

A 0 1 1 0

B 1 0 0 1

C 1 0 0 1

D 0 1 1 0

Space

$O(V^2)$

Advantages

- Fast edge lookup
- Simple implementation

Disadvantages

- Wastes memory

2. Adjacency List

$A \rightarrow B \rightarrow C$

$B \rightarrow A \rightarrow D$

$C \rightarrow A \rightarrow D$

$D \rightarrow B \rightarrow C$

Space

$O(V+E)$

Preferred for sparse graphs.

Breadth First Search (BFS)

Visits vertices level by level.

Uses

Queue

Algorithm

$BFS(G,s)$

Mark s visited

Enqueue(s)

while queue not empty

u = Dequeue()

for every neighbour v

if not visited

Mark visited

Enqueue(v)

Example

A
/\n
B C
/\n
D E

Traversal

A B C D E

Complexity

Time

$O(V+E)$

Space

$O(V)$

Applications

- Shortest path (unweighted graph)
- GPS routing
- Social networks

Depth First Search (DFS)

Visits one branch completely before backtracking.

Uses

Stack (or recursion)

Algorithm

DFS(v)

Visit(v)

for every neighbour u

 if not visited

 DFS(u)

Traversal

A B D E C

Complexity

Time

$O(V+E)$

Applications

- Cycle detection
- Topological Sort
- Strongly Connected Components

BFS vs DFS

Feature	BFS	DFS
Data Structure	Queue	Stack
Visits	Level wise	Depth wise
Shortest Path	Yes	No
Memory	More	Less
Complexity	$O(V+E)$	$O(V+E)$

Strongly Connected Components (SCC)

Applicable only for

Directed Graphs

Definition

Every vertex can reach every other vertex.

Example

$A \rightarrow B$

$\uparrow \downarrow$

$D \leftarrow C$

All are strongly connected.

Kosaraju Algorithm

Steps

1. DFS
2. Push vertices into stack
3. Reverse graph

4. DFS in stack order

Complexity

$O(V+E)$

Applications

- Social Networks
- Web Crawlers
- Circuit Analysis

Topological Sorting

Definition

Linear ordering of vertices.

Condition

Graph must be

DAG (Directed Acyclic Graph)

Example

A

↓

B

↓

C

↓

D

Topological Order

A B C D

Algorithm (DFS)

DFS(v)

Visit(v)

for every neighbour

DFS()

Push(v)

Reverse Stack

Output

Topological Order

Complexity

$O(V+E)$

Applications

- Job Scheduling
- Compiler Design
- Dependency Resolution

Divide and Conquer

Idea

Divide problem into smaller subproblems.

Solve recursively.

Combine results.

General Structure

Divide

↓

Conquer

↓

Combine

Examples

- Merge Sort
- Quick Sort
- Binary Search
- Strassen Matrix Multiplication

Control Abstraction

DAC(P)

if Small(P)

 Solve(P)

else

 Divide(P)

 DAC(P1)

 DAC(P2)

 Combine(P1,P2)

Merge Sort

Idea

Split into halves.

Merge sorted halves.

Example

38 27 43 3 9 82 10

↓

38 27 43

3 9 82 10

↓

...

↓

3 9 10 27 38 43 82

Algorithm

MergeSort(A,l,r)

if $l < r$

$mid = (l+r)/2$

MergeSort(left)

MergeSort(right)

Merge()

Merge Procedure

Compare first elements

Insert smaller

Repeat

Complexity

Recurrence

$$T(n) = 2T(n/2) + n$$

Master Theorem

$$a = 2$$

$$b = 2$$

$$f(n) = n$$

Result

$$O(n \log n)$$

Advantages

- Stable
- Guaranteed $O(n \log n)$

Disadvantages

- Extra memory required

Strassen Matrix Multiplication

Normal Matrix Multiplication

$$2 \times 2$$

Requires

8 multiplications

Strassen

Uses only

7 multiplications

Formula

$$P1=(A11+A22)(B11+B22)$$

$$P2=(A21+A22)B11$$

$$P3=A11(B12-B22)$$

$$P4=A22(B21-B11)$$

$$P5=(A11+A12)B22$$

$$P6=(A21-A11)(B11+B12)$$

$$P7=(A12-A22)(B21+B22)$$

Final Matrix

$$C11=P1+P4-P5+P7$$

$$C12=P3+P5$$

$$C21=P2+P4$$

$$C22=P1-P2+P3+P6$$

Complexity

Normal

$$O(n^3)$$

Strassen

$$T(n)=7T(n/2)+O(n^2)$$

Using Master Theorem

$O(n^{2.81})$

Advantages

- Faster for large matrices
- Reduces multiplication operations

Disadvantages

- Complex implementation
- Not efficient for very small matrices

MODULE 3

Greedy Strategy, Dynamic Programming & Backtracking

1. Greedy Strategy

A Greedy Algorithm is an algorithmic technique that makes the best possible choice at each step, hoping that these local optimal choices lead to a globally optimal solution.

Characteristics

- Makes locally optimal decisions.
- Does not reconsider previous decisions.
- Easy to implement.
- Efficient for many optimization problems.

Applications

- Fractional Knapsack
- Minimum Spanning Tree
- Dijkstra's Algorithm
- Huffman Coding
- Job Scheduling

2. Greedy Control Abstraction

Greedy(A)

Solution = $\emptyset$

while solution not complete

 Select best candidate

 if candidate is feasible

 Add candidate to solution

return solution

3. Fractional Knapsack Problem

Problem

A knapsack has capacity W .

Each object has

- Weight (W_i)
- Profit (P_i)

Objects can be divided into fractions.

Greedy Choice

Choose the object with the highest Profit/Weight ratio first.

Algorithm

Sort items in decreasing order of Profit/Weight ratio.

for each item

 if item fits

 take entire item

 else

 take required fraction

 stop

Example

Item	Profit	Weight	P/W
------	--------	--------	-----

1	60	10	6
---	----	----	---

2	100	20	5
---	-----	----	---

3	120	30	4
---	-----	----	---

Capacity = 50

Take

Item1 $\rightarrow$ 10

Item2 $\rightarrow$ 20

Remaining =20

Take 20/30 of Item3

Profit

=60+100+80

=240

Complexity

Sorting

$O(n \log n)$

Selection

$O(n)$

Overall

$O(n \log n)$

4. Minimum Cost Spanning Tree (MST)

A spanning tree that

- Connects all vertices
- Has no cycles
- Minimum total edge weight

Kruskal's Algorithm

Idea

Select smallest edge first.

Uses

Disjoint Set (Union-Find)

Algorithm

Sort edges by weight

for each edge

if adding edge does not create cycle

add edge

Example

A---1---B

| \ |

4 2 3

| \ |

C---5---D

Selected

AB

AC

BD

Complexity

Sorting

$O(E \log E)$

Union Find

$O(E \alpha(V))$

Overall

$O(E \log E)$

Prim's Algorithm

Idea

Grow MST one vertex at a time.

Uses

Priority Queue (Min Heap)

Algorithm

Select any vertex

Repeat

Choose minimum edge

Add new vertex

Until all vertices included

Complexity

Using Min Heap

$O(E \log V)$

Kruskal vs Prim

Feature	Kruskal	Prim
Starts with	Edges	Vertices
Uses	Disjoint Set	Priority Queue
Cycle Check	Yes	No
Sparse Graph	Better	Good
Dense Graph	Less Efficient	Better

5. Shortest Path Problem

Dijkstra's Algorithm

Finds shortest path from one source to all vertices.

Condition

Works only for

Positive edge weights.

Algorithm

Initialize distance

$\text{distance}[\text{source}] = 0$

Repeat

Choose minimum distance vertex

Relax adjacent edges

Until all vertices processed

Relaxation

if $\text{dist}[u] + \text{weight}(u, v) < \text{dist}[v]$

$\text{dist}[v] = \text{dist}[u] + \text{weight}$

Example

A--2--B

| |

5 1

| |

C--2--D

Shortest

A→B→D

Complexity

Array

$O(V^2)$

Priority Queue

$O(E \log V)$

Applications

GPS

Network Routing

Google Maps

Dynamic Programming

Dynamic Programming solves problems by

Breaking into overlapping subproblems

Storing solutions

Reusing results

Principle of Optimality

Bellman's Principle

"An optimal solution contains optimal solutions to its subproblems."

Dynamic Programming Control Abstraction

Solve(P)

if solution already computed

return value

else

 compute

 store

return answer

Difference

Divide & Conquer

Subproblems independent

Dynamic Programming

Subproblems overlap

Matrix Chain Multiplication

Problem

Multiply matrices in minimum number of multiplications.

Example

A1

A2

A3

A4

Different parenthesizations

$((A1A2)A3)$

$(A1(A2A3))$

Costs differ.

Recurrence

$m[i,j]$

$=\min$

$(m[i,k]$

$+m[k+1,j]$

$+p(i-1)p_kp_j)$

Algorithm

for chain length

 compute minimum cost

store answer

Complexity

Time

$O(n^3)$

Space

$O(n^2)$

Floyd-Warshall Algorithm

Finds shortest path

Between every pair of vertices.

(All Pairs Shortest Path)

Dynamic Programming Formula

$D(i,j)$

$=\min$

$(D(i,j),$

$D(i,k)+D(k,j))$

Algorithm

for k

 for i

 for j

 update shortest path

Example

Initial Matrix

0 5 INF

2 0 3

INF 1 0

Update

Using every intermediate vertex.

Complexity

Time

$O(V^3)$

Space

$O(V^2)$

Dijkstra vs Floyd-Warshall

Feature	Dijkstra	Floyd
Source	Single	All
Technique	Greedy	Dynamic Programming
Negative Edge	No	Yes (No Negative Cycle)
Complexity	$O(E \log V)$	$O(V^3)$

Backtracking

Backtracking builds solution step by step.

If solution becomes invalid

Undo previous step

Try another choice.

Control Abstraction

Backtrack(k)

if solution complete

print solution

else

for every candidate

if feasible

choose

Backtrack(k+1)

unchoose

N-Queens Problem

Problem

Place N queens on an $N \times N$ chessboard

No two queens attack each other.

Attack

- Same row
- Same column
- Same diagonal

Example

4 Queens

Q . . .

. . Q .

. Q . .

... Q

Algorithm

Place queen row by row

Check

Column

Left diagonal

Right diagonal

If safe

Place queen

Recur

Else

Backtrack

Safe Function

Check column

Check upper left diagonal

Check upper right diagonal

Complexity

Worst Case

$O(N!)$

Applications

Puzzle Solving

Scheduling

Game Playing

Constraint Satisfaction Problems

AI

Comparison of Strategies

Technique	Idea	Example
Greedy	Best choice now	Dijkstra
Divide & Conquer	Divide problems	Merge Sort
Dynamic Programming	Store subproblems	Matrix Chain
Backtracking	Try & Undo	N Queens

Time Complexity Summary

Algorithm	Complexity
Fractional Knapsack	$O(n \log n)$
Kruskal	$O(E \log E)$
Prim	$O(E \log V)$
Dijkstra	$O(E \log V)$

Algorithm	Complexity
Matrix Chain Multiplication	$O(n^3)$
Floyd-Warshall	$O(V^3)$
N-Queens	$O(N!)$

MODULE 4

Branch and Bound, Complexity Theory, Approximation & Randomized Algorithms

1. Branch and Bound

Branch and Bound (B&B) is an algorithm design technique used to solve optimization problems. It systematically explores all possible solutions but prunes (eliminates) branches that cannot produce a better solution than the current best.

Characteristics

- Searches a state-space tree.
- Uses branching to generate candidate solutions.
- Uses bounding to discard non-promising solutions.
- Guarantees an optimal solution.

Applications

- Travelling Salesman Problem (TSP)
- 0/1 Knapsack
- Job Assignment Problem

2. Branch and Bound Control Abstraction

BranchAndBound(Node)

if Node is a complete solution

 update best solution

else

 generate child nodes

 compute bound

 if bound is promising

 BranchAndBound(child)

 else

 prune child

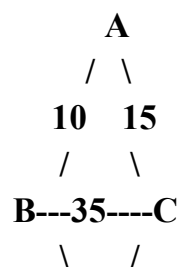
3. Travelling Salesman Problem (TSP)

Problem

A salesman must:

- Visit every city exactly once.
- Return to the starting city.
- Minimize the total travel cost.

Example



20 30

\ /

D

Goal: Find the minimum-cost Hamiltonian cycle.

Branch and Bound Algorithm

1. Start from the source city.
2. Calculate a lower bound (minimum possible remaining cost).
3. Expand the node with the smallest bound.
4. Prune nodes whose bound is greater than the current best solution.
5. Continue until the optimal tour is found.

Complexity

Worst Case:

$O(n!)$

Branch and Bound reduces the search space using pruning but does not change the worst-case complexity.

4. Complexity Theory

Tractable Problems

A problem is tractable if it can be solved in polynomial time.

Examples:

- Merge Sort
- BFS
- DFS

- Dijkstra

Complexity:

$O(n)$, $O(n^2)$, $O(n^3)$

Intractable Problems

Problems that cannot be solved efficiently (no known polynomial-time algorithm).

Examples:

- Travelling Salesman Problem
- Clique Problem
- Vertex Cover
- Hamiltonian Cycle

Usually have:

$O(2^n)$, $O(n!)$

5. Complexity Classes

Class P

Problems solvable in polynomial time.

Examples:

- Sorting
- Searching
- MST
- Shortest Path

Examples of complexity:

$O(n \log n)$, $O(n^2)$

Class NP

Problems whose solutions can be verified in polynomial time.

Examples:

- TSP (decision version)
- Clique
- Vertex Cover
- Sudoku verification

Important:

Every P problem is also an NP problem.

$P \subseteq NP$

NP-Hard

Problems at least as difficult as every NP problem.

Characteristics:

- May not belong to NP.
- Verification may not be polynomial.

Examples:

- Optimization version of TSP
- Halting Problem

NP-Complete

Problems that satisfy:

1. Belong to NP.

2. Are NP-Hard.

Examples:

- Clique
- Vertex Cover
- Hamiltonian Cycle
- SAT

Relationship

NP-Hard

NP

P

6. NP-Completeness Proof

To prove a problem is NP-Complete:

Step 1

Show it belongs to NP.

Step 2

Reduce a known NP-Complete problem to it in polynomial time.

Clique Problem

A Clique is a subset of vertices where every pair of vertices is directly connected.

Example

A ----- B

| \ |

| \ |

C ----- D

Vertices A, B, C form a clique if all are connected.

Decision Problem:

"Does a graph contain a clique of size k?"

Complexity:

NP-Complete

Vertex Cover Problem

A Vertex Cover is a set of vertices such that every edge in the graph is incident to at least one selected vertex.

Example

A-----B

\ /

\ /

C

Selecting C and A covers all edges.

Decision Problem:

"Does the graph have a vertex cover of size k?"

Complexity:

NP-Complete

Clique $\leftrightarrow$ Vertex Cover

The complement of a clique corresponds to a vertex cover in the complement graph.

This reduction is commonly used in NP-Completeness proofs.

7. Approximation Algorithms

Approximation algorithms produce solutions close to the optimal for NP-Hard problems within polynomial time.

Applications:

- Scheduling
- Routing
- Packing
- Logistics

Bin Packing Problem

Problem

Given:

- Items of different sizes.
- Bins of fixed capacity.

Objective:

Use the minimum number of bins.

Example

Capacity = 10

Items:

7 5 6 3 2

First Fit Algorithm

Steps:

1. Place an item in the first bin where it fits.
2. If no bin fits, open a new bin.

First Fit Decreasing (FFD)

Steps:

1. Sort items in decreasing order.
2. Apply First Fit.

FFD generally gives better results.

Complexity

Sorting:

$O(n \log n)$

Placement:

$O(n^2)$

8. Randomized Algorithms

Randomized algorithms use random numbers during execution.

Advantages:

- Simpler implementation.

- Good average performance.
- Avoid worst-case inputs.

Monte Carlo Algorithms

Always finish within a fixed time but may produce an incorrect answer with a small probability.

Characteristics:

- Fast.
- Small chance of error.

Examples:

- Miller-Rabin Primality Test.

Las Vegas Algorithms

Always produce the correct answer but running time is random.

Characteristics:

- Correct output guaranteed.
- Execution time varies.

Example:

- Randomized Quick Sort.

Monte Carlo vs Las Vegas

Feature	Monte Carlo	Las Vegas
Correct Answer	Not always	Always
Running Time	Fixed	Random

Feature	Monte Carlo	Las Vegas
Error Probability	Possible	None

Example

Primality Test

Randomized Quick Sort

9. Randomized Quick Sort

Idea

Instead of choosing the first or last element as pivot,

Choose a random pivot.

This avoids worst-case input patterns.

Algorithm

RandomizedQuickSort(A, low, high)

if low < high

 choose random pivot

 partition

 recursively sort left

 recursively sort right

Example

Array

9 3 5 7 1 8

Random Pivot

5

Partition

3 1 |5| 9 8 7

Recursively sort both sides.

Complexity

Best Case

$O(n \log n)$

Average Case

$O(n \log n)$

Worst Case

$O(n^2)$

Randomization makes the worst case highly unlikely.

Deterministic vs Randomized Quick Sort

Feature	Deterministic	Randomized
Pivot	Fixed	Random
Worst Case	Frequent on sorted input	Rare
Average Time	$O(n \log n)$	$O(n \log n)$
Practical Performance	Less Robust	More Robust

Summary of Time Complexities

Algorithm	Complexity
Branch & Bound (TSP)	$O(n!)$
Clique Verification	$O(n^2)$
Vertex Cover Verification	$O(E)$
First Fit Bin Packing	$O(n^2)$
First Fit Decreasing	$O(n \log n) + O(n^2)$
Randomized Quick Sort (Average)	$O(n \log n)$
Randomized Quick Sort (Worst)	$O(n^2)$

Comparison of Complexity Classes

Class	Meaning	Examples
P	Polynomial-time solvable	Merge Sort, BFS, DFS
NP	Solution verifiable in polynomial time	Clique, Vertex Cover
NP-Hard	At least as hard as NP problems	Optimization TSP
NP-Complete	In NP and NP-Hard	Clique, Vertex Cover, SAT